Informatique et systèmes de communication ISC

Haute Ecole d'Ingénierie
Hochschule für Ingenieurwissenschaften

Data Processing and Visualization

C3. Tabular 1
Data cleaning and preprocessing

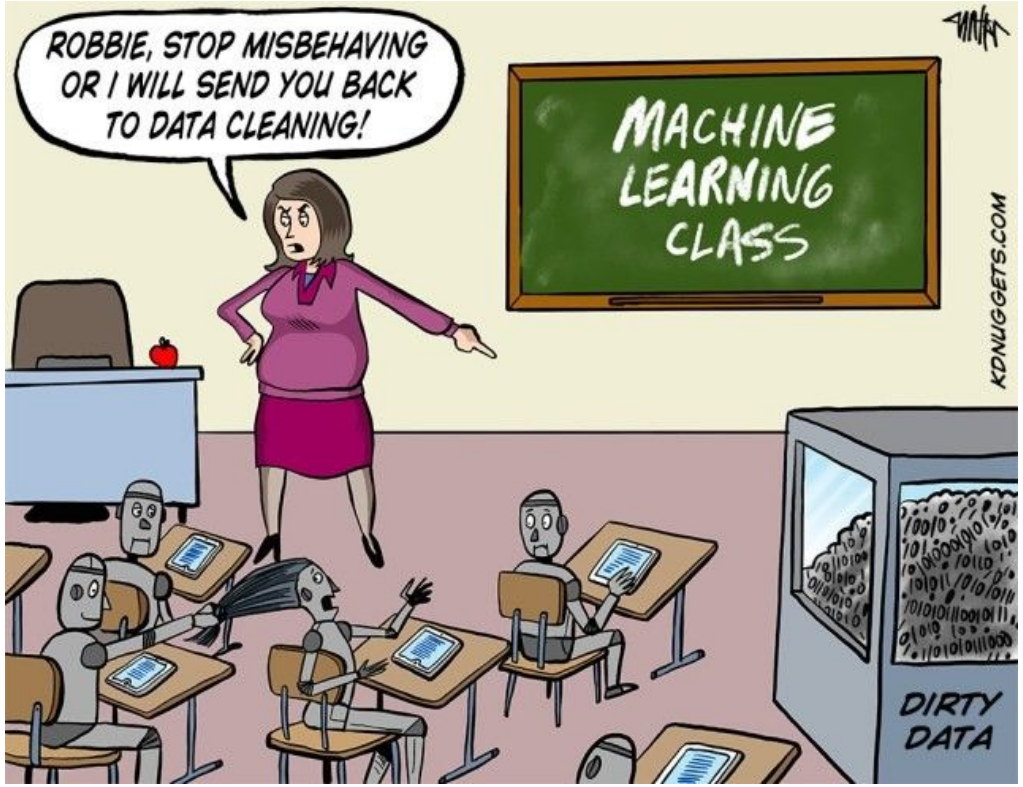
Vincent Andrearczyk
vincent.andrearczyk@hevs.ch

René Schumann
rene.schumann@hevs.ch

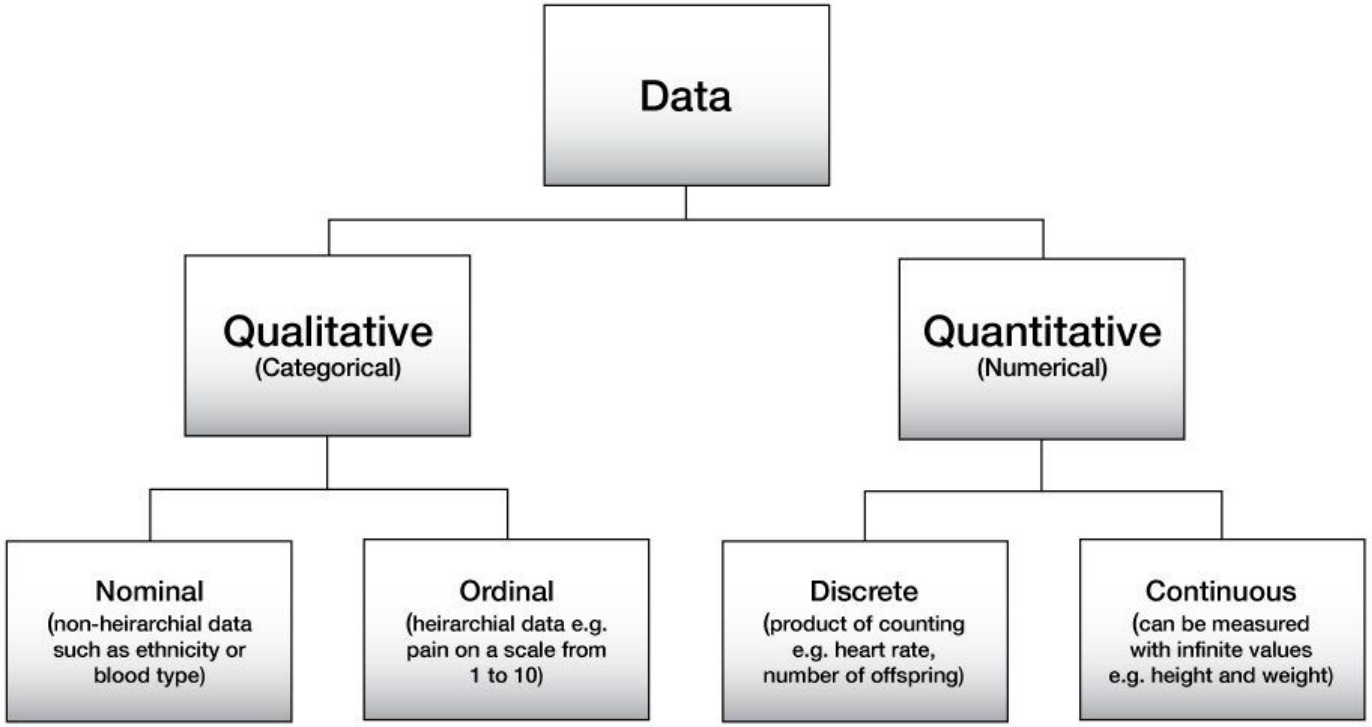
Background

Background

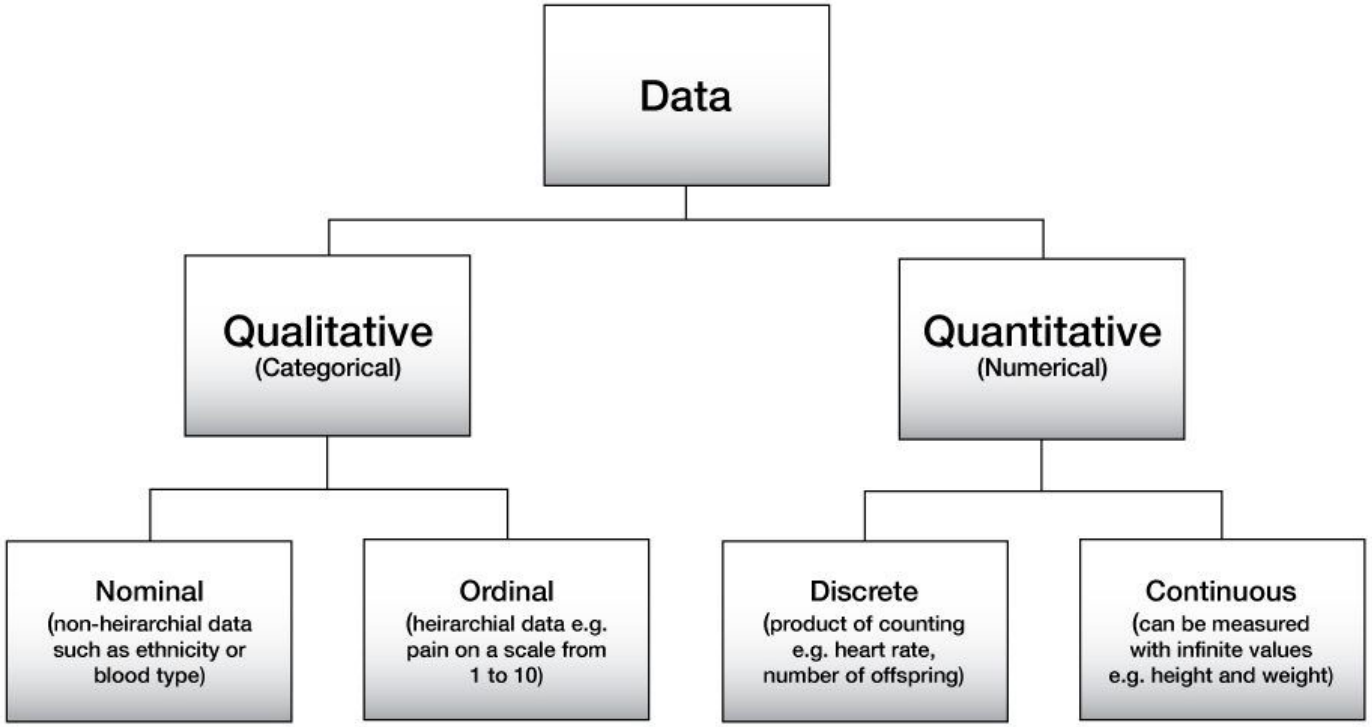
- For various reasons (acquisition problems, human error, etc.) data is not always clean
  - Big impact on analyses, decisions, ML models performance …
  - Cleaning is necessary
- Pre-processing also crucial to analyses:
  - Aggregation (summarizing)
  - Filtering (choosing small parts)
  - Transformation (e.g. normalization)



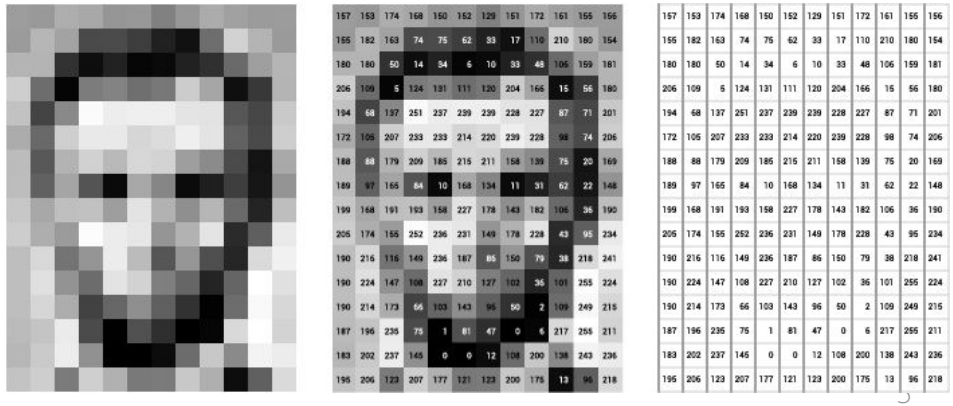
Background - Types of tabular data



Background - Types of tabular data



- Tabular = Structured data
- Images, audio, video, text = Unstructured data



Background - Types of tabular data

- Categorical data: Variables are composed of a limited number of possible values.
  - **Nominal data**: categories without any specific order or ranking
    - Nationality: swiss, italian, french, etc.
    - Eye color: blue, green, brown etc.
    - Binary data e.g. Heads and tails (head, tail), has cancer (yes, no)
  - **Ordinal data**: categories with a meaningful order or ranking
    - Coffee strength: light, medium, strong
    - Cancer stage: 1, 2, 3, 4

Background - Types of tabular data

- Categorical data: Variables are composed of a limited number of possible values.
  - **Nominal data**: categories without any specific order or ranking
    - Nationality: swiss, italian, french, etc.
    - Eye color: blue, green, brown etc.
    - Binary data e.g. Heads and tails (head, tail), has cancer (yes, no)
  - **Ordinal data**: categories with a meaningful order or ranking
    - Coffee strength: light, medium, strong
    - Cancer stage: 1, 2, 3, 4

Marital status ? Car brand ? Customer satisfaction ? Programming languages ? Movie ratings ? Clothing sizes ?

Background - Types of tabular data

- Numerical data: present in number form.
  - **Discrete data**: can only take certain values.
    - Number of students in a class
    - Number of cars in a parking lot
  - **Continuous data**:
    - Height of a person
    - Weight of an object

Important terms

Database: Information organized into rows and columns e.g. spreadsheet or table. System designed for storing and managing data efficiently
Dataset: A collection of data (simpler, static, single analysis)
Distribution (of data): The way data values are spread or distributed in a dataset
Feature/variable/attribute/predictor (used interchangeably): measurable piece of data that can be used for analysis: Name, Age, Size, ...
Filter: Select a subset of the data
Relationship: A connection or correlation between two or more variables
Sort: Ordering data (e.g. largest to smallest)
Summarize: Deriving one value from a series of other values to produce a summary statistic (e.g. mean)

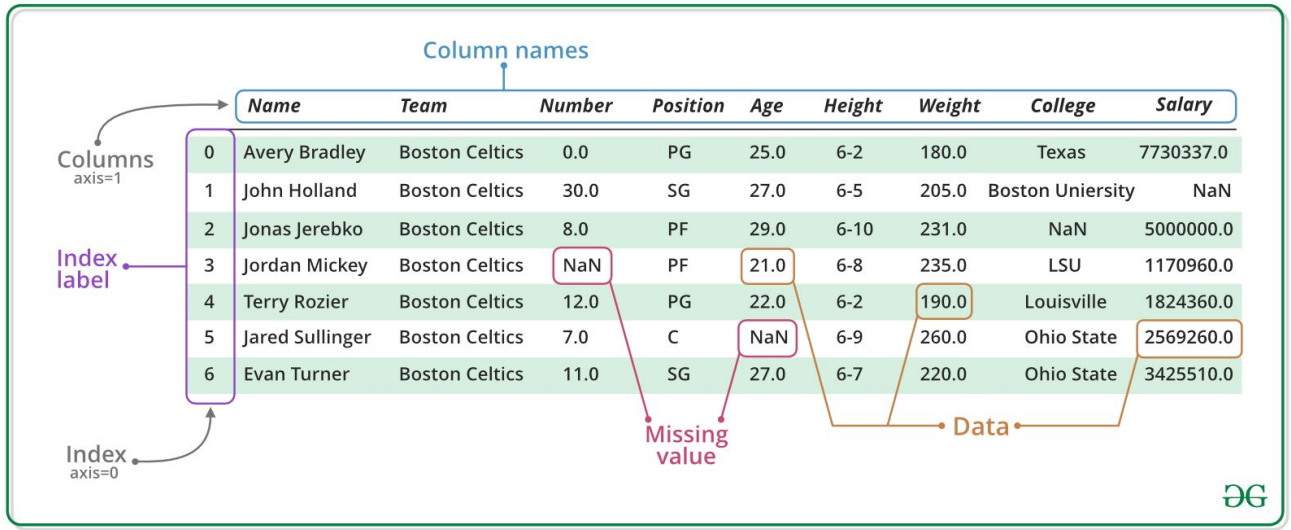
Data Processing and Visualization Python Libraries

Python libraries:

- NumPy wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies many high-level mathematical functions that operate on these arrays and matrices
- Matplotlib library for creating static, animated, and interactive visualizations
  - **Pandas** is built on top of *matplotlib* for data visualization and *NumPy* for mathematical operations. It acts as a wrapper over these libraries
  - Seaborn is a data visualization library based on *matplotlib*
- Scikit-learn provides many unsupervised and supervised learning algorithms. It is built upon *NumPy*, *pandas*, and *Matplotlib*
- Plotly is a library for data visualization. It produces interactive graphs and can be embedded on websites

Data Processing and Visualization Python Libraries

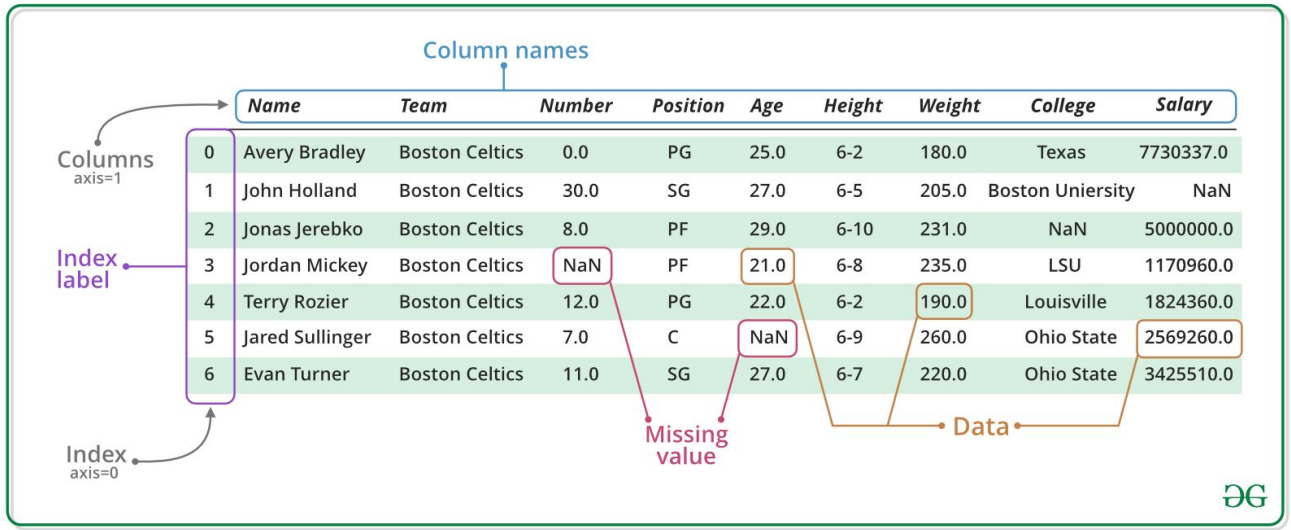
- Basics of Pandas: https://pandas.pydata.org/docs/user_guide/10min.html
  - Particularly important to start:
    - Structure of a df
    - Quick df exploration: df.head(), df.describe(), df.info(), df[‘col’].unique(), df.keys(),
    - Slice data: df.loc[], df.iloc[], etc.
    - Read the doc



	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Data Processing and Visualization Python Libraries

- Basics of Pandas: https://pandas.pydata.org/docs/user_guide/10min.html
  - Particularly important to start:
    - Structure of a df
    - Quick df exploration: df.head(), df.describe(), df.info(), df[‘col’].unique(), df.keys(),
    - Slice data: df.loc[], df.iloc[], etc.
    - Read the doc



	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

→ Experiment with dataframes using this tutorial (Done)

1. Introduction to Data Cleaning and Preprocessing

1. Introduction to Data Cleaning and Preprocessing

- Data cleaning:
  - Fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset
  - Why it happens: many reasons
    - Combining multiple data sources
      - Duplicates
      - Different units of measure
    - Deliberate lie, deliberately not answered
    - Information not available
    - Incorrect measurement
    - Incorrect data entry, etc.
  - Why it is needed:
    - Incorrect data → Unreliable outcomes (visualizations, decisions, ML algorithms)

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - 1. Data quality assessment
  - 2. Remove duplicate or irrelevant observations
  - 3. Identifying and fixing structural errors (naming conventions, typos, or incorrect capitalization)
  - 4. Handling missing values
  - 5. Detecting and removing outliers (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - **1. Data quality assessment**
  - 2. Remove duplicate or irrelevant observations
  - 3. Identifying and fixing structural errors (naming conventions, typos, or incorrect capitalization)
  - 4. Handling missing values
  - 5. Detecting and removing outliers (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

- 1. Data quality assessment
  - Relevance (useful to the analysis/business/research question)
  - Accuracy (close to true values)
  - Completeness (all required data is known)
  - Uniformity (same units of measure)
  - Timeliness (not obsolete measurements)
  - …

→ Plan data cleaning

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - 1. Data quality assessment
  - **2. Remove duplicate or irrelevant observations**
  - 3. Identifying and fixing structural errors (naming conventions, typos, or incorrect capitalization)
  - 4. Handling missing values
  - 5. Detecting and removing outliers (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

- 2. Remove duplicate or irrelevant observations
  - Do the duplicates represent the real world? No → Remove

1. Introduction to Data Cleaning and Preprocessing

- 2. Remove duplicate or irrelevant observations
  - Do the duplicates represent the real world? No → Remove

```
# Cars in a garage
df.head(10)
```

✓ 0.0s

	Brand	Year	Color
0	Toyota	2019	Blue
1	Ford	2020	Red
2	Toyota	2019	Blue
3	Chevrolet	2021	Silver
4	Ford	2018	Black
5	Ford	2017	Black
6	Toyota	2020	Blue

```
# Students results
df.head(10)
```

✓ 0.0s

	Student ID	Age	Grade
0	101	20	5.0
1	102	22	4.5
2	101	20	5.0
3	103	21	4.5
4	104	20	5.5
5	105	23	4.0
6	106	19	5.5

1. Introduction to Data Cleaning and Preprocessing

- 2. Remove duplicate or irrelevant observations
  - Do the duplicates represent the real world? No → Remove

```
# Cars in a garage
df.head(10)
```

✓ 0.0s

	Brand	Year	Color
0	Toyota	2019	Blue
1	Ford	2020	Red
2	Toyota	2019	Blue
3	Chevrolet	2021	Silver
4	Ford	2018	Black
5	Ford	2017	Black
6	Toyota	2020	Blue

```
# Students results
df.head(10)
```

✓ 0.0s

	Student ID	Age	Grade
0	101	20	5.0
1	102	22	4.5
2	101	20	5.0
3	103	21	4.5
4	104	20	5.5
5	105	23	4.0
6	106	19	5.5

```
df.drop_duplicates()
```

✓ 0.0s

	Student ID	Age	Grade
0	101	20	5.0
1	102	22	4.5
3	103	21	4.5
4	104	20	5.5
5	105	23	4.0
6	106	19	5.5

1. Introduction to Data Cleaning and Preprocessing

- 2. Remove duplicate or irrelevant observations
  - Do the duplicates represent the real world? No → Remove
  - Warning: Duplicates may vary slightly: More difficult to detect

Record 1:
Name: John Doe
Email: john.doe@email.com
Address: 123 Main Street
Phone: 555-1234

Record 2:
Name: John Doe
Email: john.doe@email.com
Address: 124 Main Street
Phone: 555-1234

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - 1. Data quality assessment
  - 2. Remove duplicate or irrelevant observations
  - **3. Identifying and fixing structural errors** (naming conventions, typos, or incorrect capitalization)
  - 4. Handling missing values
  - 5. Detecting and removing outliers (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

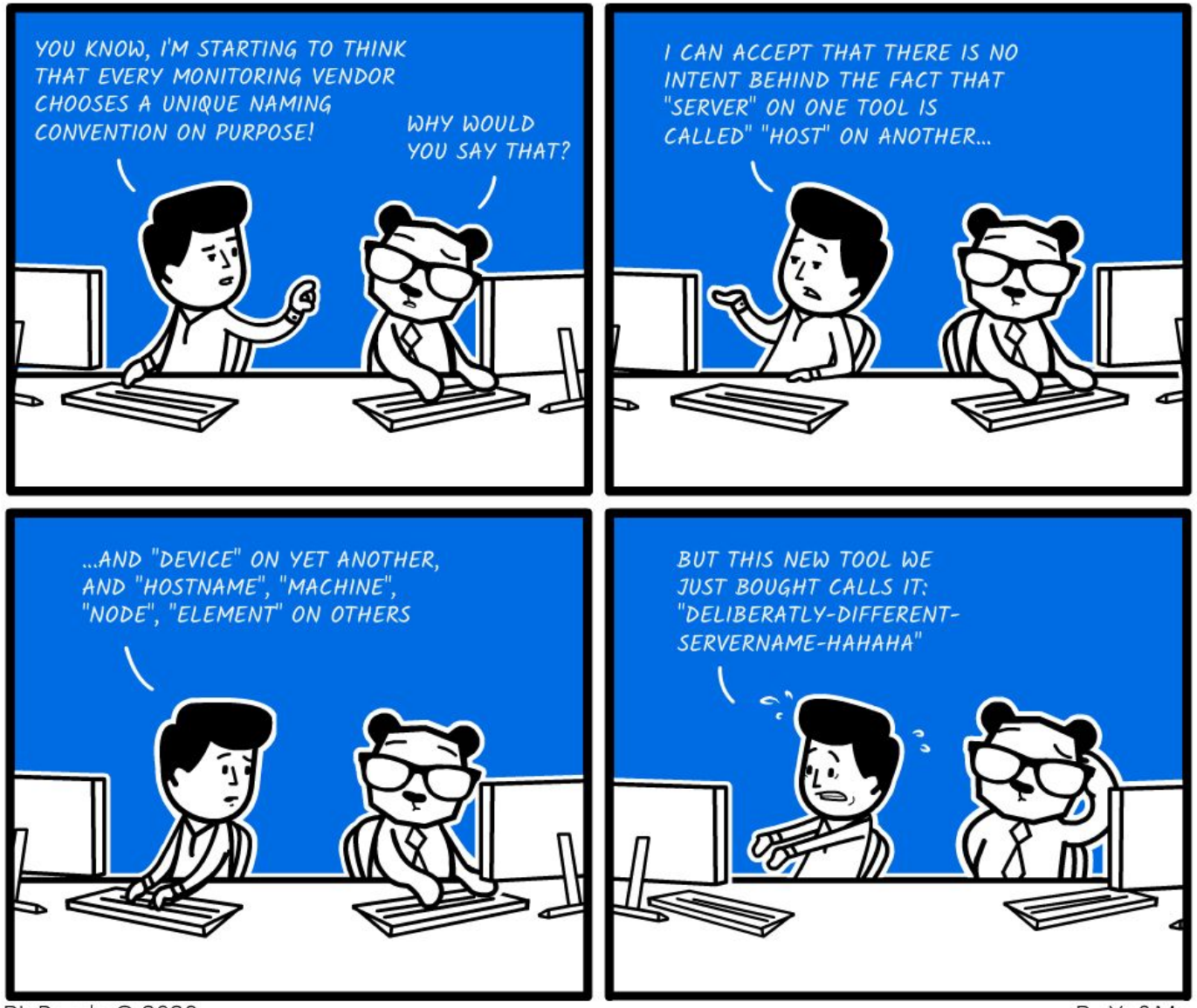
- 3. Identifying and fixing structural errors
  - Naming conventions
  - Typos
  - Incorrect capitalization
  - Different date formats (mm/dd/yyyy, dd/mm/yyyy etc.)

	Brand	Model	Price	In Stock
0	Apple	Macbook	1400	Yes
1	apple	Mcabook	900	Y
2	HP	HP	n/a	Yes
3	Apple	Macbook pro	2000	No
4	Apple	Macbook	1800	4
5	Asus	ZenBook	NA	0
6	Apple	macbook	-	Yes
7	Acer	Swift	1200	no
8	Apple	Macbook	na	Yes

1. Introduction to Dat

- 3. Identifying and fixing structu
  - Naming conventions
  - Typos
  - Incorrect capitalization
  - Different date formats (mm

	Brand	Model	Price	In Stock
0	Apple	Macbook	1400	Yes
1	apple	Mcabook	900	Y
2	HP	HP	n/a	Yes
3	Apple	Macbook pro	2000	No
4	Apple	Macbook	1800	4
5	Asus	ZenBook	NA	0
6	Apple	macbook	-	Yes
7	Acer	Swift	1200	no
8	Apple	Macbook	na	Yes



BigPanda © 2020 By Yo&Ma

1. Introduction to Data Cleaning and Preprocessing

- 3. Identifying and fixing structural errors
  - Naming conventions
  - Typos
  - Incorrect capitalization
  - Different date formats (mm/dd/yyyy, dd/mm/yyyy etc.)

```
df.head(5)
```

✓ 0.0s

	Brand	Year	Color
0	Toyota	2019	Blue
1	Ford	2020	Redd
2	Toyota	2019	Blue
3	Chevrolet	2021	Red
4	Ford	2018	Black

1. Introduction to Data Cleaning and Preprocessing

- 3. Identifying and fixing structural errors
  - Naming conventions
  - Typos
  - Incorrect capitalization
  - Different date formats (mm/dd/yyyy, dd/mm/yyyy etc.)

```
df.head(5)
```

✓ 0.0s

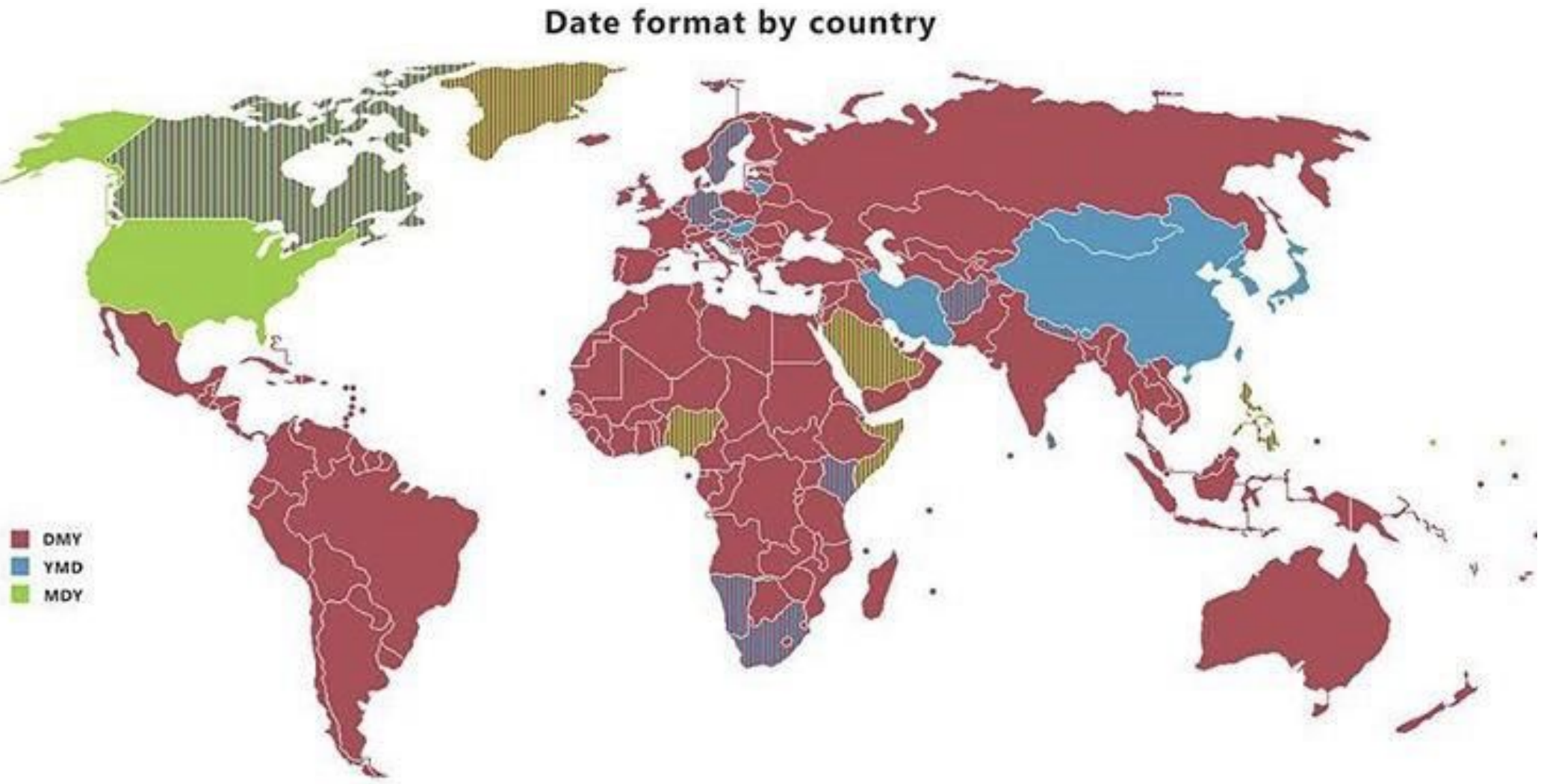
	Brand	Year	Color
0	Toyota	2019	Blue
1	Ford	2020	Redd
2	Toyota	2019	Blue
3	Chevrolet	2021	Red
4	Ford	2018	Black

```
df['Color'] = df['Color'].replace({'Redd': 'Red'})
df.head(5)
```

✓ 0.0s

	Brand	Year	Color
0	Toyota	2019	Blue
1	Ford	2020	Red
2	Toyota	2019	Blue
3	Chevrolet	2021	Red
4	Ford	2018	Black

1. Introduction to Data Cleaning and Preprocessing



And also
20.01.2025
20/01/2025
20.1.2025
20012025
...

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - 1. Data quality assessment
  - 2. Remove duplicate or irrelevant observations
  - 3. Identifying and fixing structural errors (naming conventions, typos, or incorrect capitalization)
  - **4. Handling missing values**
  - 5. Detecting and removing outliers (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values (numerical and categorical)
  - No universal method: Case-by-case decision
  - Find missing data: df.isna(), df.isna().sum(), df.notna(), df.info()
  - Assess the reasons for missing data
    - Survey respondent skipped question
    - Lost/corrupted file
    - etc.

1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values (numerical and categorical)
  - No universa
  - Find missin
  - Assess the
    - Survey r
    - Lost/cor
    - etc.

```python
# Airbnb data
df['security_deposit'].unique()
```

✓ 0.0s

```
array([nan, '$100.00', '$1,000.00', '$700.00', '$150.00', '$500.00',
       '$300.00', '$400.00', '$1,500.00', '$250.00', '$200.00', '$750.00',
       '$140.00', '$350.00', '$600.00', '$95.00', '$5,000.00', '$125.00',
       '$800.00', '$499.00', '$399.00', '$450.00', '$995.00', '$2,500.00',
       '$175.00', '$225.00', '$550.00', '$115.00', '$120.00', '$99.00',
       '$149.00', '$780.00', '$2,000.00', '$1,200.00', '$360.00',
       '$375.00', '$199.00', '$109.00', '$449.00', '$195.00', '$295.00',
       '$185.00', '$134.00', '$650.00', '$180.00', '$1,900.00', '$220.00'],
      dtype=object)
```

Warning: Sometimes not real missing data

1. Introduction to Data Cleaning and Preprocessing

https://www.missingdata.nl/missing-data/missing-data-mechanisms/mnar/

- 4. **3 Types of missing values: MCAR, MAR, MNAR**
  - **MCAR - Missing completely at random**
    All the variables and observations have the same probability of being missing (often unrealistic)

Complete data	
Age	IQ score
25	133
26	121
29	91
30	105
30	110
31	98
44	118
46	93
48	141
51	104
51	116
54	97

Incomplete data	
Age	IQ score
25	
26	121
29	91
30	
30	110
31	
44	118
46	93
48	
51	
51	116
54	

1. Introduction to Data Cleaning and Preprocessing

https://www.missingdata.nl/missing-data/missing-data-mechanisms/mnar/

- 4. **3 Types of missing values:**
  - MCAR - Missing completely at random
    All the variables and observations have the same probability of being missing (often unrealistic)
  - **MAR - Missing at random** (should be "missing conditionally at random")
    Probability of the value being missing is related to other variables in the dataset

Complete data	
Age	IQ score
25	133
26	121
29	91
30	105
30	110
31	98
44	118
46	93
48	141
51	104
51	116
54	97

Incomplete data	
Age	IQ score
25	
26	
29	
30	
30	
31	
44	118
46	93
48	141
51	104
51	116
54	97

1. Introduction to Data Cleaning and Preprocessing

https://www.missingdata.nl/missing-data/missing-data-mechanisms/mnar/

- 4. **3 Types of missing values:**
  - MCAR - Missing completely at random
    All the variables and observations have the same probability of being missing (often unrealistic)
  - MAR - Missing at random (should be "missing conditionally at random")
    Probability of the value being missing is related to other variables in the dataset
  - **MNAR- Missing not at random**
    Probability of missing values of a variable are related to the values of that variable itself.
    → More difficult to handle. <u>Assume MAR when possible</u>

Complete data	
Age	IQ score
25	133
26	121
29	91
30	105
30	110
31	98
44	118
46	93
48	141
51	104
51	116
54	97

Incomplete data	
Age	IQ score
25	133
26	121
29	
30	
30	110
31	
44	118
46	
48	141
51	
51	116
54	

1. Introduction to Data Cleaning and Preprocessing

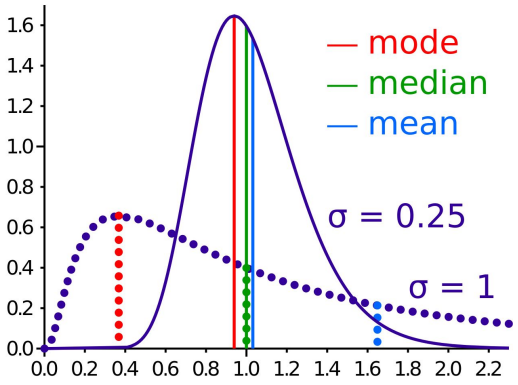
- 4. **3 Types of missing values:**
  - MCAR - Missing completely at random
    All the variables and observations have the same probability of being missing (often unrealistic)
  - MAR - Missing at random (should be “missing conditionally at random”)
    Probability of the value being missing is related to other variables in the dataset
  - MNAR- Missing not at random
    Probability of missing values of a variable are related to the values of that variable itself
  - **How to know ?**
    - Common sense (what do we know about the data)
    - Statistical test

1. Introduction to Data Cleaning and Preprocessing

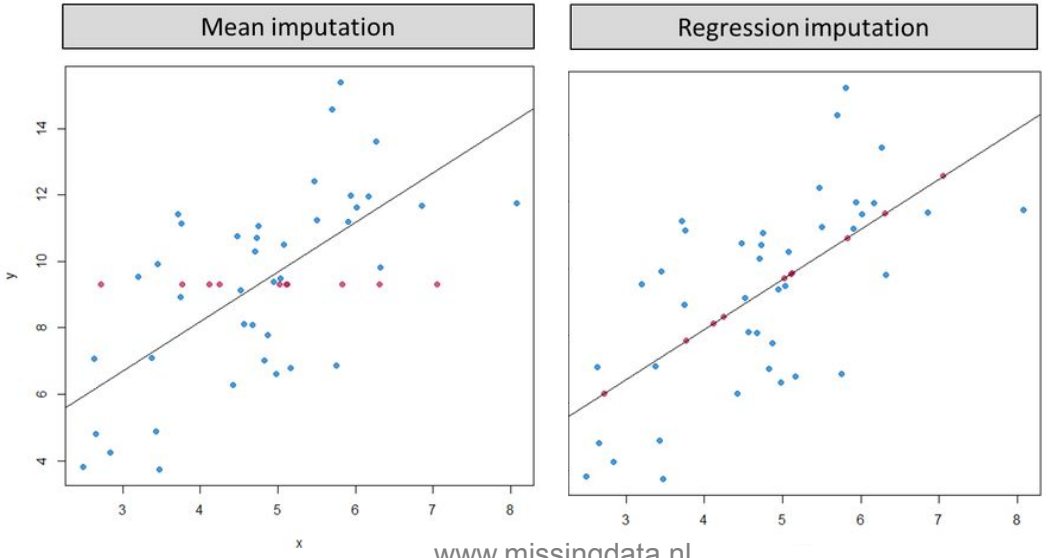
- 4. Handling missing values
  - **Deletion**
    - Complete case analysis (all cases with missing scores deleted)
    - Pairwise deletion (analysis-by-analysis basis → different sample sizes. Eg for univariate analyses)

1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values
  - Deletion
    - Complete case analysis (all cases with missing scores deleted)
    - Pairwise deletion (analysis-by-analysis basis → different sample sizes)
  - **Single imputation** (mean, median, mode, regression/classification, matching, last observation carried forward)



Comparison of mean, median and mode of two distributions with different skewness.



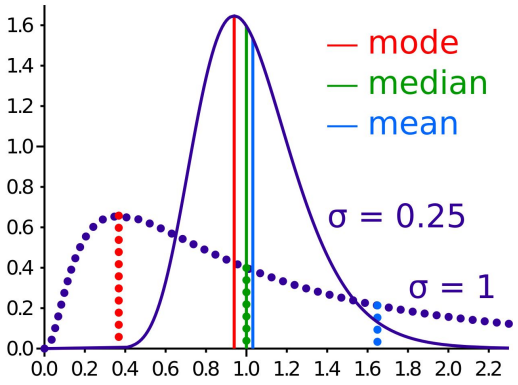
www.missingdata.nl

Age	Height	Weight	Weight_matching_imp
10	135	31.0	31
11	144	NaN	31
22	178	80.0	80
28	168	69.0	69
42	172	77.0	77
45	176	81.0	81
52	169	73.0	73

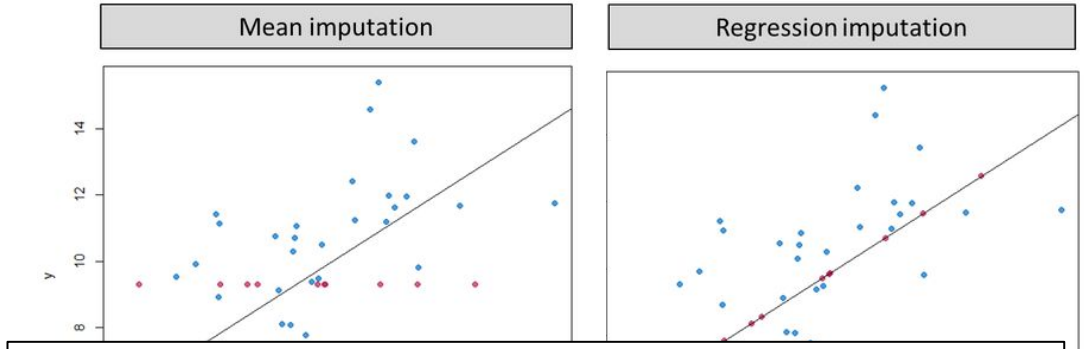
PatientID	Months	Glucose	Glucose_locf_imp
001	0	82.0	82.0
001	2	86.0	86.0
001	4	90.0	90.0
001	6	NaN	90.0
001	8	92.0	92.0

1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values
  - Deletion
    - Complete case analysis (all cases with missing scores deleted)
    - Pairwise deletion (analysis-by-analysis basis → different sample sizes)
  - **Single imputation** (mean, median, mode, regression/classification, matching, last observation carried forward)



Comparison of mean, median and mode of two distributions with different skewness.



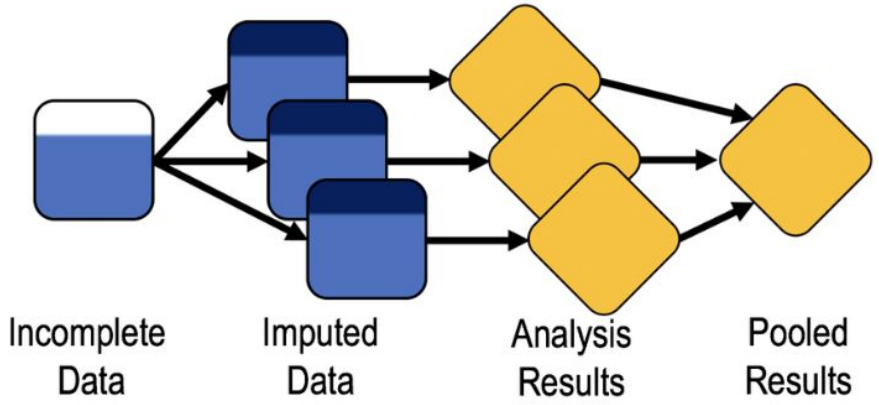
→ Risk of biased result :
mode for categorical -> imbalance;
mean for num -> reduced variance

Age	Height	Weight	Weight_matching_imp
10	135	31.0	31
11	144	NaN	31
22	178	80.0	80
28	168	69.0	69
42	172	77.0	77
45	176	81.0	81
52	169	73.0	73

PatientID	Months	Glucose	Glucose_locf_imp
001	0	82.0	82.0
001	2	86.0	86.0
001	4	90.0	90.0
001	6	NaN	90.0
001	8	92.0	92.0

1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values
  - Deletion
    - Complete case analysis (all cases with missing scores deleted)
    - Pairwise deletion (analysis-by-analysis basis → different sample sizes)
  - Single imputation (mean, median, mode, regression/classification, matching, last observation carried forward)
  - **Multiple imputation** (good for MAR)
    Imputation process <u>repeated multiple times</u> resulting in multiple imputed datasets. Accounts for imputation uncertainty.



1. Introduction to Data Cleaning and Preprocessing

- 4. Handling missing values

Method	MCAR	MAR	Notes
Deletion	✅ (if low % missing)	❌	Reduces sample size, biased for MAR
Mean Imputation	✅ (if low % missing)	⚠️ (risk of bias)	Distorts relationships
Regression Imputation	⚠️ (not useful)	✅	Can underestimate variance
Multiple Imputation	✅ (if high % missing)	✅ Best choice	Computationally expensive

1. Introduction to Data Cleaning and Preprocessing

- How to clean data: No exact/universal way
  - 1. Data quality assessment
  - 2. Remove duplicate or irrelevant observations
  - 3. Identifying and fixing structural errors (naming conventions, typos, or incorrect capitalization)
  - 4. Handling missing values
  - **5. Detecting and removing outliers** (e.g. improper data-entry)

1. Introduction to Data Cleaning and Preprocessing

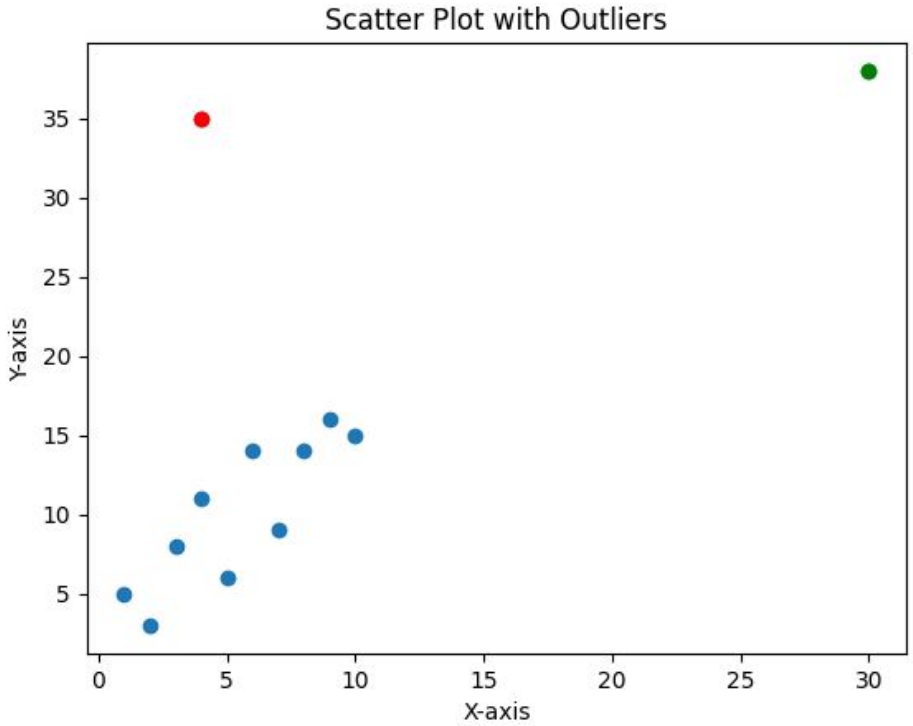
► 5. Detecting and removing outliers

1. Introduction to Data Cleaning and Preprocessing

- 5. Detecting and removing outliers
  - Outlier: observation that lies an abnormal distance from other values
  - **True outliers** represent natural data variations: **should be retained**
  - **Other outliers** (e.g. measurement error, data entry error, etc.) **should be removed**

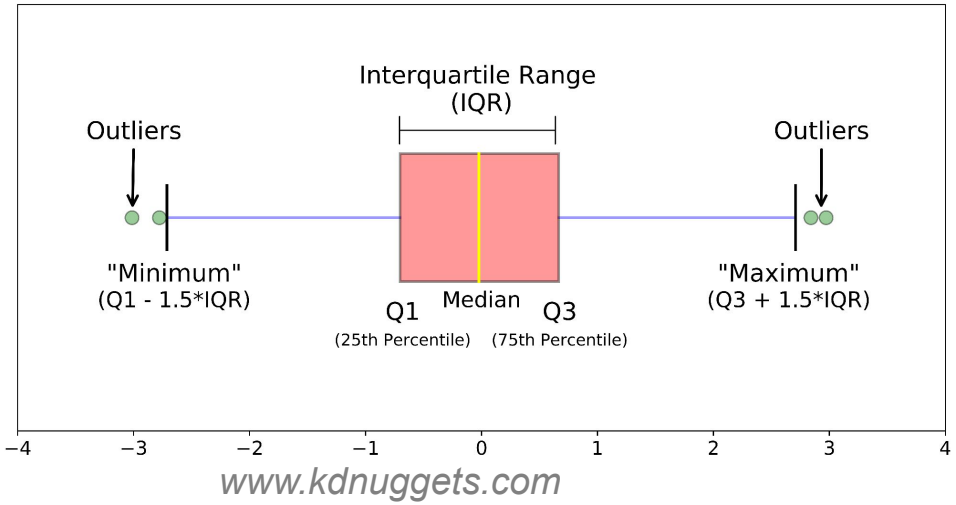
1. Introduction to Data Cleaning and Preprocessing

- 5. Detecting and removing outliers
  - Outlier: observation that lies an abnormal distance from other values
  - **True outliers** represent natural data variations: **should be retained**
  - **Other outliers** (e.g. measurement error, data entry error, etc.) **should be removed**
  - Univariate outlier
  - Multivariate outlier



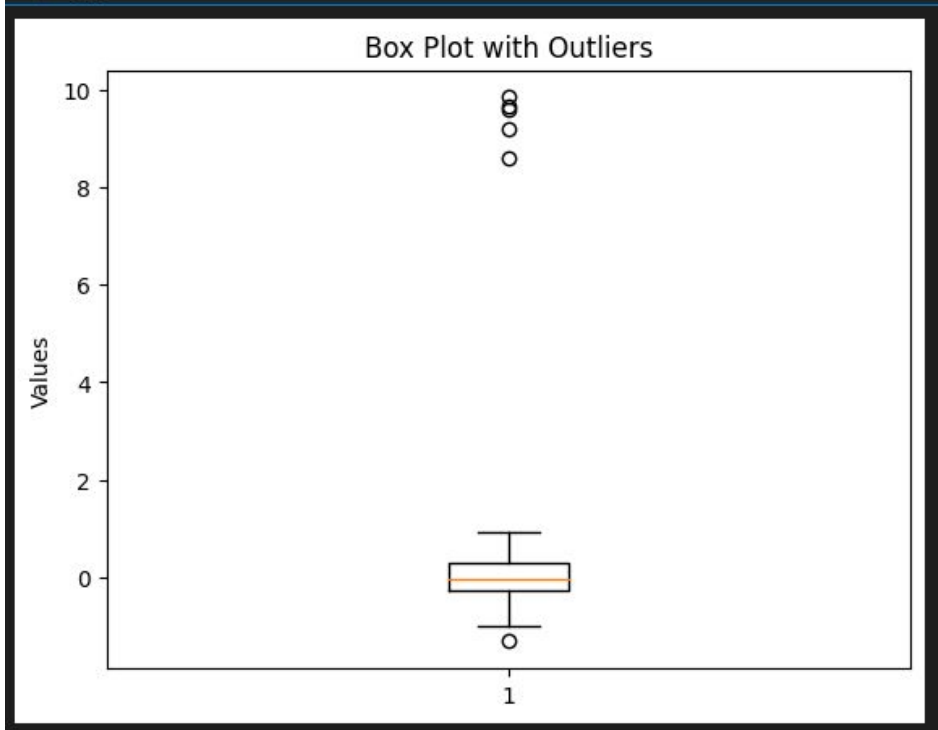
1. Introduction to Data Cleaning and Preprocessing

- 5. **Detecting** and removing outliers
  - Visualization: Boxplot (details in C3)



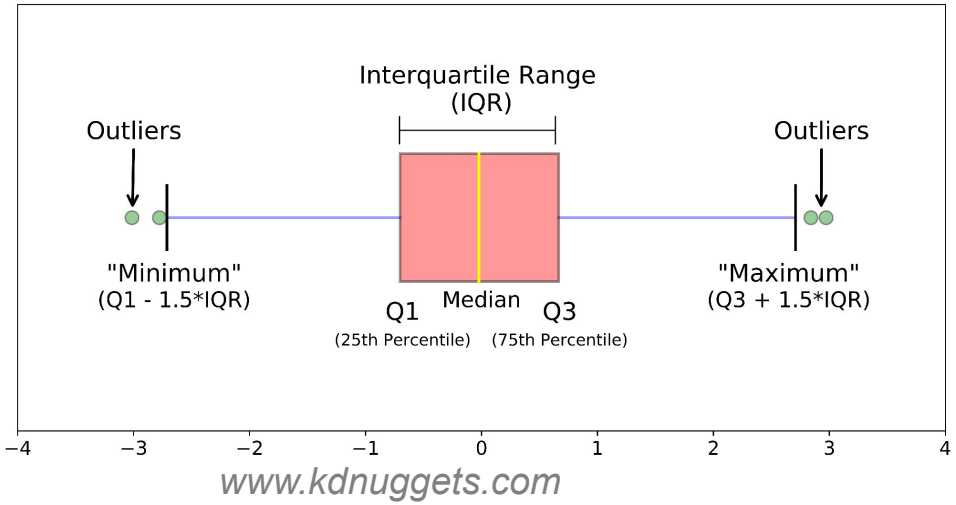

```python
# Generate some random data with outliers
np.random.seed(42)
data = np.concatenate([np.random.normal(0, 1, 100), np.random.normal(10, 1, 5)])

# Create a box plot to visualize outliers
plt.boxplot(data)
plt.title('Box Plot with Outliers')
plt.ylabel('Values')
plt.show()
```

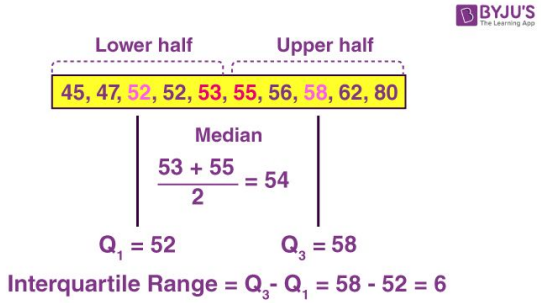
✓ 0.1s



1. Introduction to Data Cleaning and Preprocessing

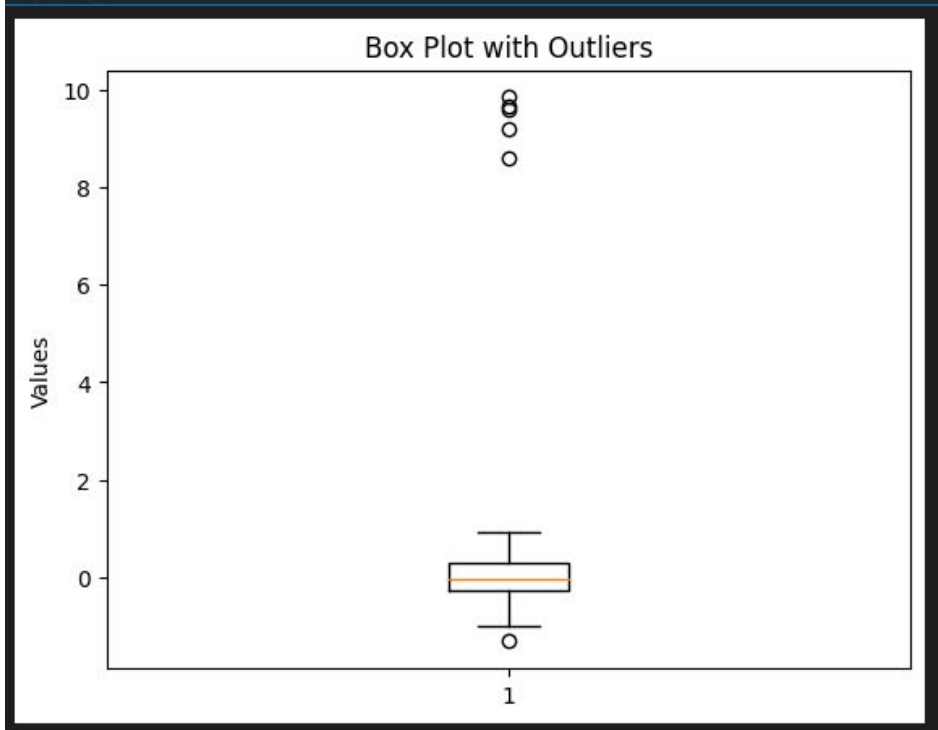
- 5. **Detecting** and removing outliers
  - Visualization: Boxplot (details in C3)
  - IQR






```python
# Generate some random data with outliers
np.random.seed(42)
data = np.concatenate([np.random.normal(0, 1, 100), np.random.normal(10, 1, 5)])

# Create a box plot to visualize outliers
plt.boxplot(data)
plt.title('Box Plot with Outliers')
plt.ylabel('Values')
plt.show()
```

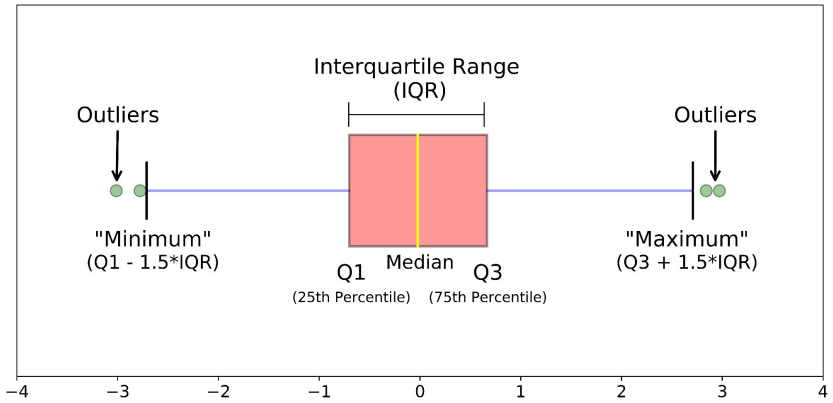
✓ 0.1s



1. Introduction to Data Cleaning and Preprocessing

- 5. **Detecting** and removing outliers
  - Visualization: Boxplot (details in C3)
  - IQR

1. Sort data from low to high
2. Identify the first quartile (Q1), the median, and the third quartile (Q3)
   `df['col'].quantile(0.25)` `df['col'].quantile(0.75)`
3. Calculate IQR = Q3 – Q1
4. Calculate upper fence = Q3 + (1.5 * IQR)
5. Calculate lower fence = Q1 – (1.5 * IQR)
6. Use fences to find outliers (all values that fall outside the fences)

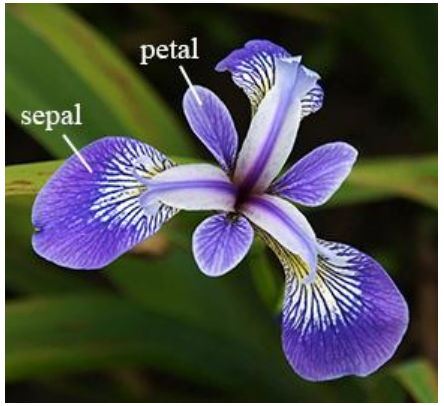


1. Introduction to Data Cleaning and Preprocessing

- 5. Detecting and **removing** outliers
  - True or other outliers ? Common sense, sometimes difficult to tell
  - Remove: df.drop()

Exercises

	measurement.number	sepal.length	sepal.width	petal.length	petal.width	variety
0	1	5.1	3.5	1.4	0.2	Setosa
1	2	4.9	3.0	1.4	0.2	Setosa
2	3	4.7	3.2	1.3	0.2	Setosa
3	4	4.6	3.1	1.5	0.2	Setosa
4	5	5.0	3.6	1.4	0.2	Setosa



2.1 Missing values, outliers, and duplicates.

Dataset: iris_to_clean.csv

a) Load and quickly inspect the dataset.

b) Check if the dataset contains duplicates. Remove them if any.

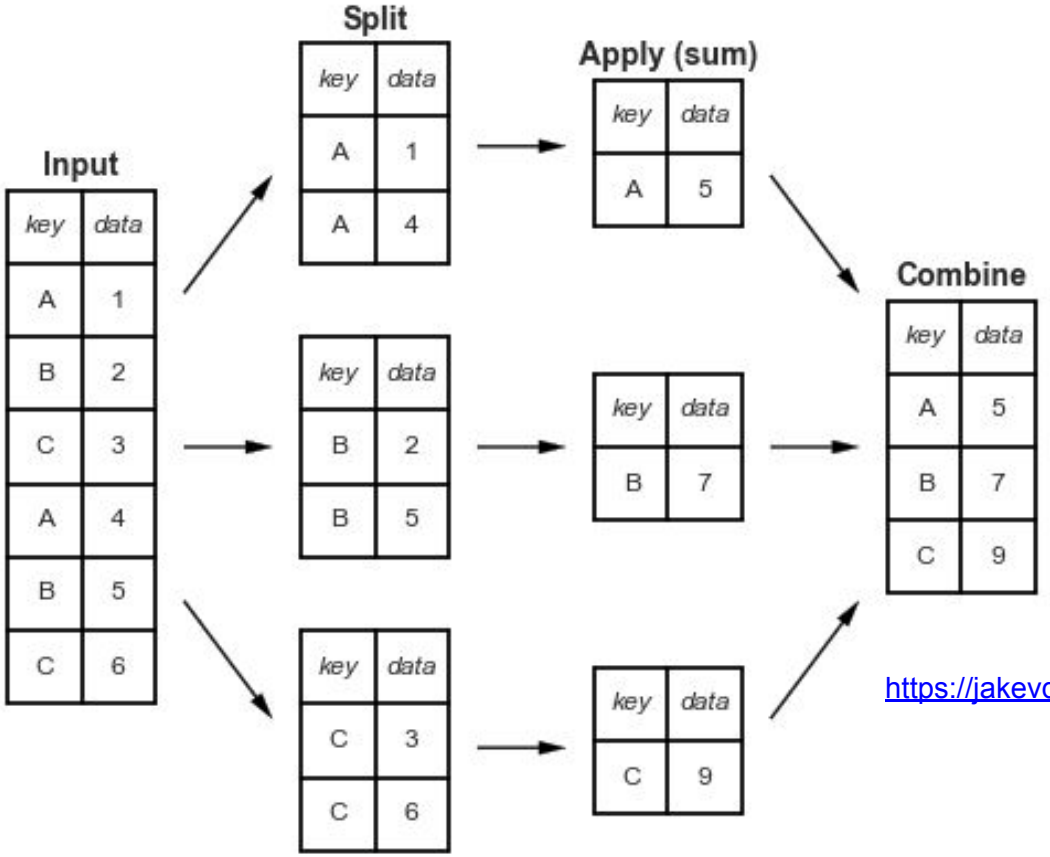
c) Check if the dataset contains missing values. Sort them with most missing on top and compute percentages. Print rows with missing values. Choose and apply a method to handle the missing values.

d) Check if the dataset contains outliers. Remove if any non-true outlier.

2. Aggregation in Data

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
  - E.g. group average instead of information from an individual



https://jakevdp.github.io/PythonDataScienceHandbook

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
  - E.g. group average instead of information from an individual

Animal	Max Speed	Wing Span
Falcon	380.0	54.0
Falcon	340.0	47.0
Falcon	380.0	52.0
Falcon	325.0	42.0
Parrot	24.0	48.0
Parrot	20.0	50.0
Parrot	35.0	49.0
Parrot	26.0	49.0

How can we aggregate/analyze ?

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables: *Splitting* the data into groups based on some criteria

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables: *Splitting* the data into groups based on some criteria

```
df
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.sort_values(by=['Animal'])
```

✓ 0.0s

?

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables: *Splitting* the data into groups based on some criteria

```
df
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.sort_values(by=['Animal'])
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
2	Falcon	370.0
1	Parrot	24.0
3	Parrot	26.0

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables: *Splitting* the data into groups based on some criteria

```
df
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.sort_values(by=['Animal'])
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
2	Falcon	370.0
1	Parrot	24.0
3	Parrot	26.0

```
df.sort_values(by=['Max Speed'])
```

✓ 0.0s

?

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables: *Splitting* the data into groups based on some criteria

```
df
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.sort_values(by=['Animal'])
```

✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
2	Falcon	370.0
1	Parrot	24.0
3	Parrot	26.0

```
df.sort_values(by=['Max Speed'])
```

✓ 0.0s

	Animal	Max Speed
1	Parrot	24.0
3	Parrot	26.0
2	Falcon	370.0
0	Falcon	380.0

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, stdev, median)

```
df
```
✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.groupby('Animal').mean()
```
✓ 0.0s

```
df.groupby('Animal').count()
```
✓ 0.0s

```
df.groupby('Max Speed').count()
```
✓ 0.0s

```
df.groupby('Max Speed').mean()
```
⊗ 0.2s

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, stdev, median)

```
df
```
✓ 0.0s

	Animal	Max Speed
0	Falcon	380.0
1	Parrot	24.0
2	Falcon	370.0
3	Parrot	26.0

```
df.groupby('Animal').mean()
```
✓ 0.0s

	Max Speed
Animal	
Falcon	375.0
Parrot	25.0

```
df.groupby('Animal').count()
```
✓ 0.0s

	Max Speed
Animal	
Falcon	2
Parrot	2

```
df.groupby('Max Speed').count()
```
✓ 0.0s

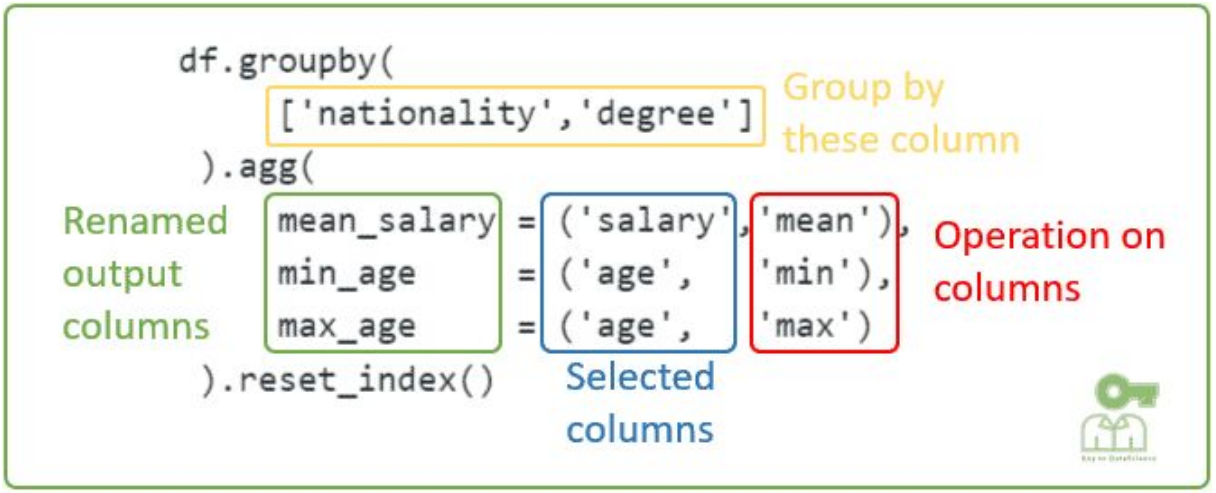
	Animal
Max Speed	
24.0	1
26.0	1
370.0	1
380.0	1

```
df.groupby('Max Speed').mean()
```
⊗ 0.2s

```
TypeError: Could not convert Parrot to num
```

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, stdev, median)



keytodatascience.com

```
df.groupby('Animal').agg(
                        mean_maxspeed=('Max Speed','mean'),
                        count=('Max Speed','count'),
                        min_maxspeed=('Max Speed','min'),
                        max_maxspeed=('Max Speed','max'),
                        ).reset_index()
```

✓ 0.0s

	Animal	mean_maxspeed	count	min_maxspeed	max_maxspeed
0	Falcon	375.0	2	370.0	380.0
1	Parrot	25.0	2	24.0	26.0

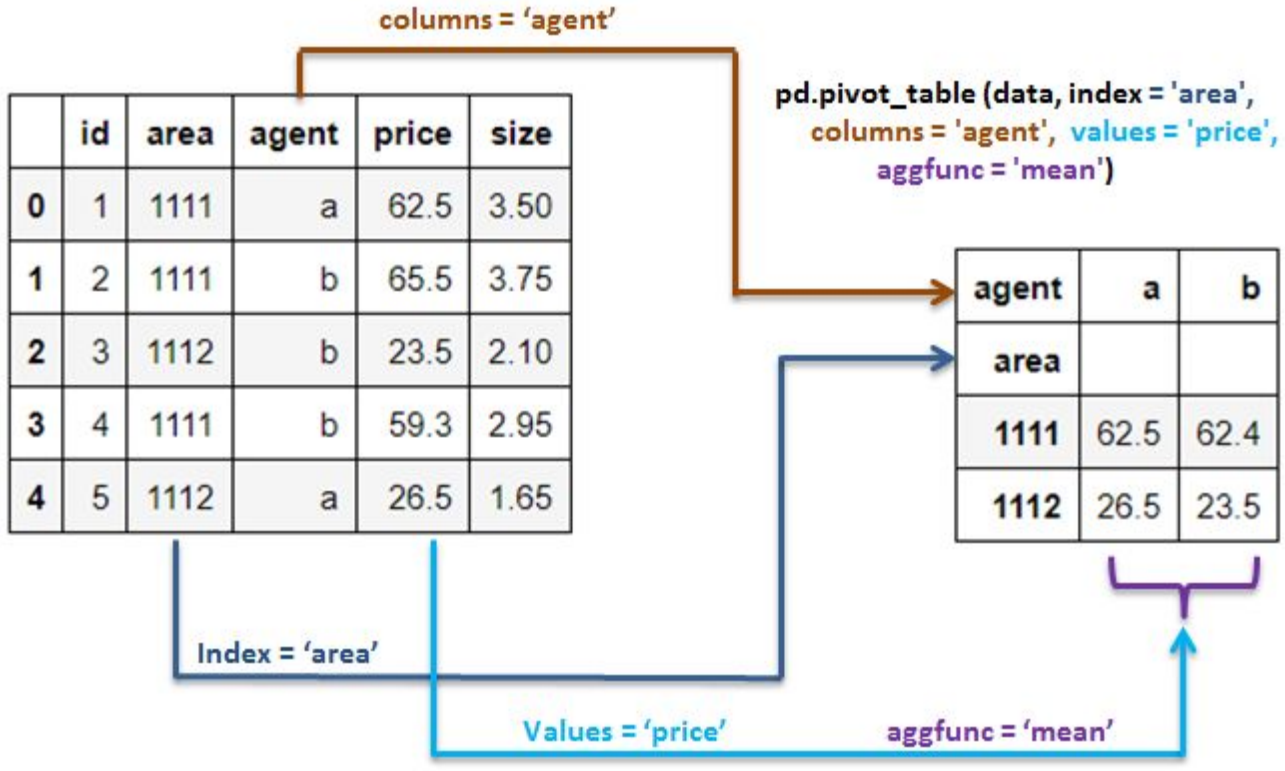
2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, median)
- Pivot tables: Quickly grouping, summarizing, aggregating and calculating statistics

pd.pivot_table()

2. Aggregation in Data

- Data aggregation: Process of gatherin summary
- Grouping data using categorical/nume
- Applying aggregation functions (sum, median)
- Pivot tables: Quickly grouping, summa statistics



	id	area	agent	price	size
0	1	1111	a	62.5	3.50
1	2	1111	b	65.5	3.75
2	3	1112	b	23.5	2.10
3	4	1111	b	59.3	2.95
4	5	1112	a	26.5	1.65

agent	a	b
area		
1111	62.5	62.4
1112	26.5	23.5

towardsdatascience.com

pd.pivot_table()
index: Specifies the **row**-level **grouping**
column: Specifies the **column** level **grouping**
values: Numerical values to summarize
aggfunc: How to **aggregate** the function, e.g. sum, mean etc.

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, median)
- Pivot tables: Quickly grouping, summarizing, aggregating and calculating statist

index
column
values
aggfunc

df

✓ 0.0s

	Date	Category	Value
0	2024-01-01	A	12
1	2024-01-01	A	5
2	2024-01-01	B	15
3	2024-01-01	B	15
4	2024-01-02	A	15
5	2024-01-02	B	25
6	2024-01-03	A	12

2. Aggregation in Data

- Data aggregation: Process of gathering raw data and expressing it in a summary
- Grouping data using categorical/numerical variables
- Applying aggregation functions (sum, mean, count, min, max, mad, median)
- Pivot tables: Quickly groupi
statist

index
column
values
aggfunc

```
df
```

✓ 0.0s

	Date	Category	Value
0	2024-01-01	A	12
1	2024-01-01	A	5
2	2024-01-01	B	15
3	2024-01-01	B	15
4	2024-01-02	A	15
5	2024-01-02	B	25
6	2024-01-03	A	12

```
pd.pivot_table(df, values='Value', index='Date', columns='Category', aggfunc='sum')
```

✓ 0.0s

Category	A	B
Date		
2024-01-01	17.0	30.0
2024-01-02	15.0	25.0
2024-01-03	12.0	NaN

```
pd.pivot_table(df, values='Value', index='Category', columns='Date', aggfunc='sum')
```

✓ 0.0s

Date	2024-01-01	2024-01-02	2024-01-03
Category			
A	17.0	15.0	12.0
B	30.0	25.0	NaN

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cog
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Ewallet	522
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	Cash	76

Exercises

2.2 Perform aggregation operations on the supermarket_sales dataset

Context: The growth of supermarkets in most populated cities are increasing and market competitions are also high. The dataset is one of the historical sales of supermarket company which has recorded data in 3 different branches for 3 months. Attribute information available here.

1 row = purchases of 1 customer

a) Compute the sum, mean and count of "quantity", grouped by payment method (with df.groupby). Comment on the results

b) Obtain the same mean quantity grouped by payment with a simple pivot table

c) Compute the count per Customer type and per City with a pivot table

supermarket_sales attributes:

- **Invoice ID**: Unique identifier for each transaction.
- **Branch**: The branch location of the supermarket (e.g., Yangon, Naypyitaw, Mandalay).
- **City**: The city in which the supermarket branch is located.
- **Customer Type**: Indicates whether the customer is a 'Member' or 'Normal'.
- **Gender**: Gender of the customer.
- **Product Line**: The category of the product sold (e.g., Health & Beauty, Electronic Accessories, Home & Lifestyle).
- **Unit Price**: Price per unit of the product.
- **Quantity**: Number of items purchased.
- **Tax 5%**: Calculated tax amount on the transaction at a 5% rate.
- **Total**: Total amount for the transaction including tax.
- **Date**: Date of the transaction.
- **Time**: Time of the transaction.
- **Payment**: Payment method used (e.g., Cash, Ewallet, Credit card).
- **COGS**: Cost of goods sold, representing the raw cost of the products.
- **Gross Margin Percentage**: Fixed percentage of profit for each sale (4.7619%).
- **Gross Income**: Profit earned from the transaction.
- **Rating**: Customer satisfaction rating (out of 10).

3. Filtering Data

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
  - Example: We want to choose an activity (sports, museum, game, etc.) for ISC students. We have data for favourite activity of swiss citizens.
    How to filter ?

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
  df.loc(), ==, !=, >, >=, .isin(), &, .str.contains(), .str.startswith(), etc.

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
  df.loc(), ==, !=, >, >=, .isin(), &, .str.contains(), .str.startswith(), etc.

```
df.loc[(df['Cost'] >= 1000) & (df['Cost'] < 2000)]
```

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
  df.loc(), ==, !=, >, >=, .isin(), &, .str.contains(), .str.startswith(), etc.

```
df.loc[(df['Cost'] >= 1000) & (df['Cost'] < 2000)]
```

What happens ? Boolean indexing:
condition is a binary mask (True, False, ...)

```
condition = (df['Cost'] >= 1000) & (df['Cost'] < 2000)
df.loc[condition]
```

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
- Handling time-based date: Example 1

```
df
```

✓ 0.0s

	Car	Date_of_Purchase
0	BMW	2021-07-10
1	Lexus	2021-08-12
2	Audi	2021-06-17
3	Mercedes	2021-03-16
4	Jaguar	2021-05-19
5	Bentley	2021-08-22

```
df.loc[df["Date_of_Purchase"] > "2021-07-15"]
```

✓ 0.0s

	Car	Date_of_Purchase
1	Lexus	2021-08-12
5	Bentley	2021-08-22

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
- Handling time-based date: Example 2

```
df
```

✓ 0.0s

	Car	Date_of_Purchase
0	BMW	10/07/2021
1	Lexus	12/08/2021
2	Audi	17/06/2021
3	Mercedes	16/03/2021
4	Jaguar	19/05/2021
5	Bentley	22/08/2021

```
df.loc[df["Date_of_Purchase"] > "15/07/2021"]
```

✓ 0.0s

	Car	Date_of_Purchase
2	Audi	17/06/2021
3	Mercedes	16/03/2021
4	Jaguar	19/05/2021
5	Bentley	22/08/2021

Wrong, why?

3. Filtering Data

- Data filtering: Choosing a smaller part of the dataset for analysis or visualization
- Filtering based on conditions
- Handling time-based date: Example 2 - pd.to_datetime()

```
df
```

✓ 0.0s

	Car	Date_of_Purchase
0	BMW	10/07/2021
1	Lexus	12/08/2021
2	Audi	17/06/2021
3	Mercedes	16/03/2021
4	Jaguar	19/05/2021
5	Bentley	22/08/2021

```
df.loc[df["Date_of_Purchase"] > "15/07/2021"]
```

✓ 0.0s

	Car	Date_of_Purchase
2	Audi	17/06/2021
3	Mercedes	16/03/2021
4	Jaguar	19/05/2021
5	Bentley	22/08/2021

Wrong, why?

```
df['Date_of_Purchase'] = pd.to_datetime(df['Date_of_Purchase'], format='%d/%m/%Y')
df
```

✓ 0.0s

	Car	Date_of_Purchase
0	BMW	2021-07-10
1	Lexus	2021-08-12
2	Audi	2021-06-17
3	Mercedes	2021-03-16
4	Jaguar	2021-05-19
5	Bentley	2021-08-22

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cog
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	13:08	Ewallet	522
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	10:29	Cash	76

Exercises

2.3 Filter the data based on conditions.

We use again the supermarket_sales dataset.

a) We want to conduct the study only in the Yangon city (for the entire exercise). Filter the data.

b) Look at some transactions with low ratings (<=4), some with high rating (>=9.9). Then filter these out for the rest of the exercise

c) Calculate the Total price in a new column, computed as Unit price * quantity + Tax 5%. Compare with the "Total" column.

d) A 10% discount on the total price was given to the members (a column in the df specifies which customers are members). Calculate that price in a new column. Use df.apply with a lambda function with condition.

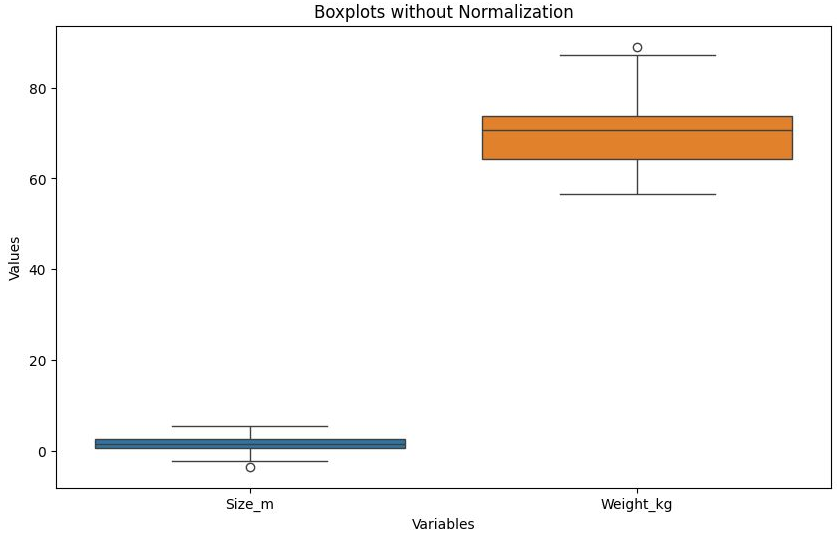
4. Transformation of Data

4. Transformation of Data

- Data transformation
  - Feature scaling: standardization and normalization
  - Feature engineering
  - Encoding categorical variables

4. Transformation of Data

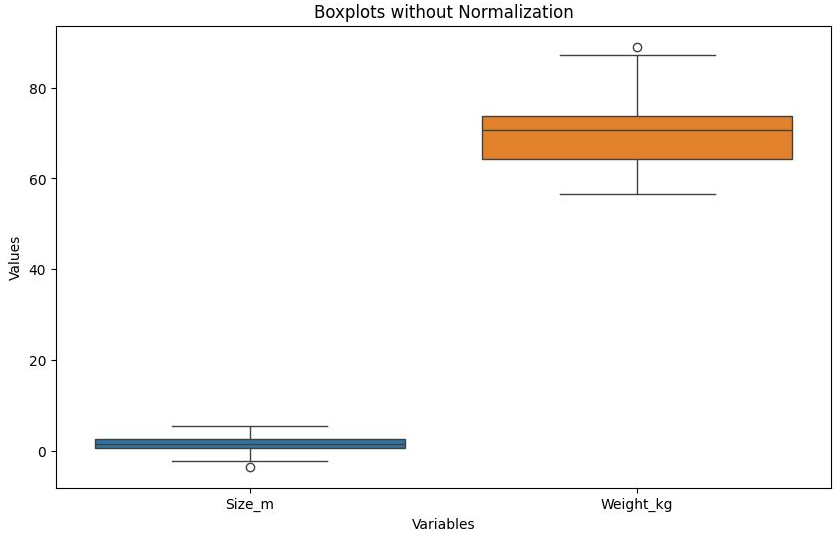
- Feature scaling
  - What is feature scaling? Normalizing the range of features in a dataset
  - Why?
    - Real-world datasets often contain features that vary in degrees of magnitude, range, and units
    - For comparison or to develop ML models, we need features on the same scale.
  - How ? Standardization or Normalization

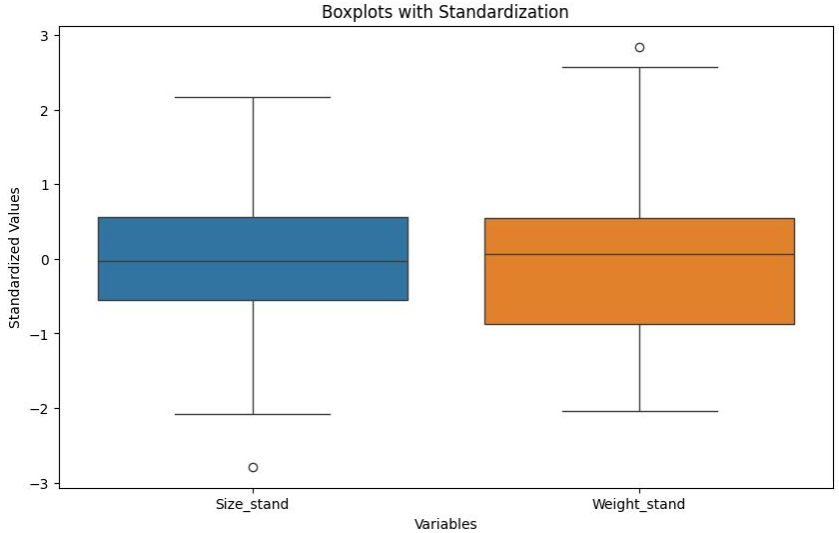


Example: Size (m) and weight (kg)

4. Transformation of Data

- Feature scaling
  - What is feature scaling? Normalizing the range of features in a dataset
  - Why?
    - Real-world datasets often contain features that vary in degrees of magnitude, range, and units
    - For comparison or to develop ML models, we need features on the same scale.
  - How ? Standardization or Normalization





4. Transformation of Data

- Feature scaling: **Normalization** (Min-max)

$$x' \in [0, 1]$$

bounded by value range

4. Transformation of Data

- Feature scaling: Normalization (Min-max)

$$\boxed{X' = \frac{X - X_{min}}{X_{max} - X_{min}}}$$

$$x' \in [0, 1]$$

min(X) → 0
max(X) → 1

4. Transformation of Data

- Feature scaling: Normalization (Min-max)

$$x' \in [0, 1]$$

$$\boxed{X' = \frac{X - X_{min}}{X_{max} - X_{min}}}$$

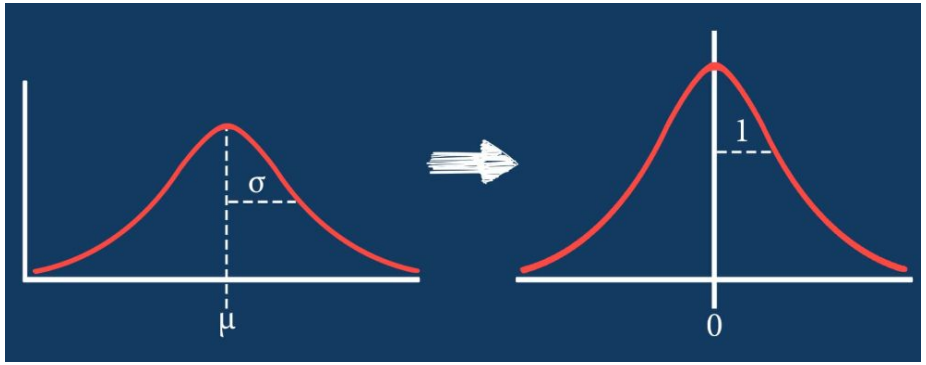
min(X) → 0
max(X) → 1

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler()
data_minmax = min_max_scaler.fit_transform(data)
```

```
min_value = data.min()
max_value = data.max()
data_minmax = (data - min_value) / (max_value - min_value)
```

4. Transformation of Data

- Feature scaling: **Standardization** (Z-score)



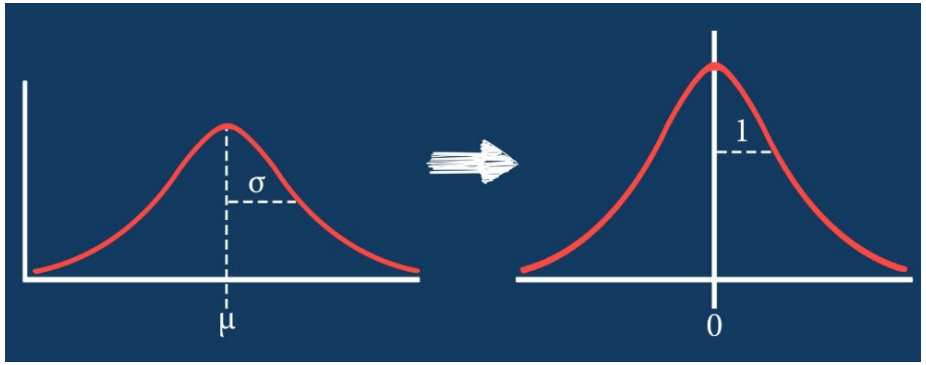
towardsdatascience.com

$$\boxed{X' = \frac{X - \mu}{\sigma}}$$

$$\mu' = 0$$
$$\sigma' = 1$$

4. Transformation of Data

- Feature scaling: Standardization (Z-score)



towardsdatascience.com

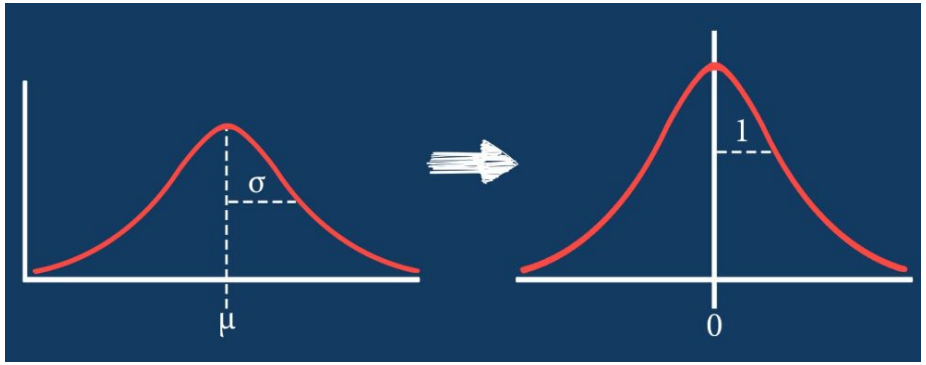
$$\mu' = 0$$
$$\sigma' = 1$$

$$\boxed{X' = \frac{X - \mu}{\sigma}}$$

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data)
```

4. Transformation of Data

- Feature scaling: Standardization (Z-score)



towardsdatascience.com

$$X' = \frac{X - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data_standardized = scaler.fit_transform(data)
```

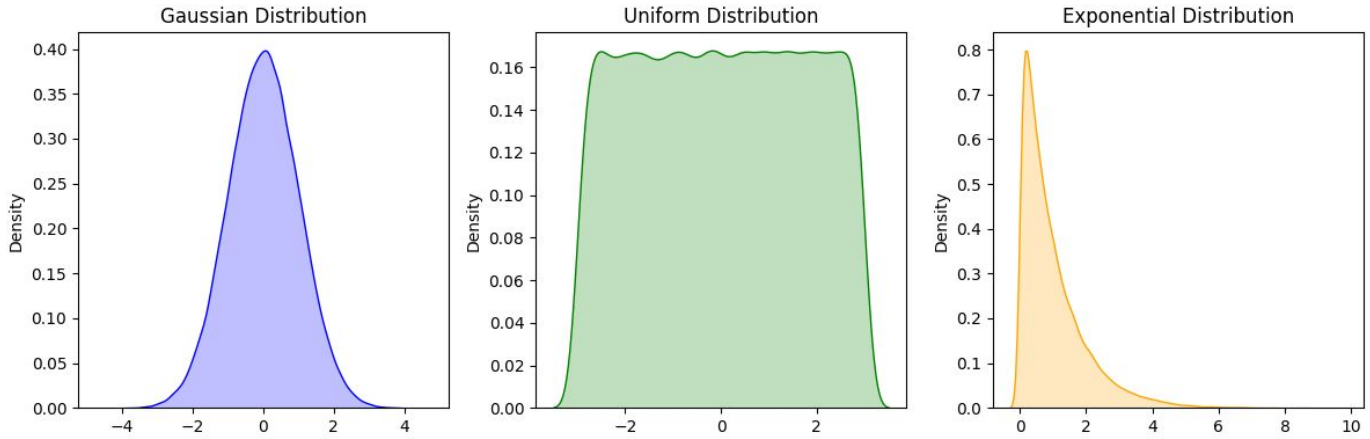
$$\mu' = 0$$
$$\sigma' = 1$$

Not bounded by value range

4. Transformation of Data

- Feature scaling: **Normalization or Standardization** ?
  - Data is Gaussian → Standardization
  - Not Gaussian → Normalization

  (in general, not strictly)

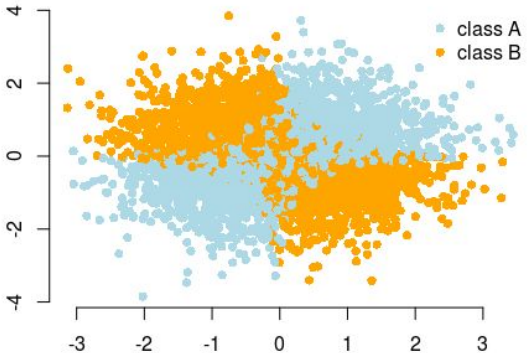


  - Normalization is affected by outliers (compress values into small range)

4. Transformation of Data

- **Feature engineering**
  - Create new features based on existing ones
  - For visualization and for ML
  - Example 1: Combine body mass index (BMI) with blood pressure measurements to create a cardiovascular health score.

  - Example 3: For ML (e.g. feat1 x feat2)



  - Example 2: Competitors in the area for opening a new grocery store

```
df
```

✓ 0.0s

	Coop	Migros	Denner
0	2	1	1
1	1	1	0
2	1	0	1
3	0	1	0
4	1	2	0

```
df['Competitors'] = df['Coop']+df['Migros']+df['Denner']
df
```

✓ 0.0s

	Coop	Migros	Denner	Competitors
0	2	1	1	4
1	1	1	0	2
2	1	0	1	2
3	0	1	0	1
4	1	2	0	3

4. Transformation of Data

medium.com/aiskunks

- **Encoding categorical variables** (for ML)

Most ML algorithms work only with numerical data

- Ordinal encoding
  - Each category assigned a label
  - Ok when categories have a Natural Ordering
- One-Hot Encoding
  - Avoids arbitrary order of categorical values

Ordinal Encoding

Grades
A
B
C
D
Fail

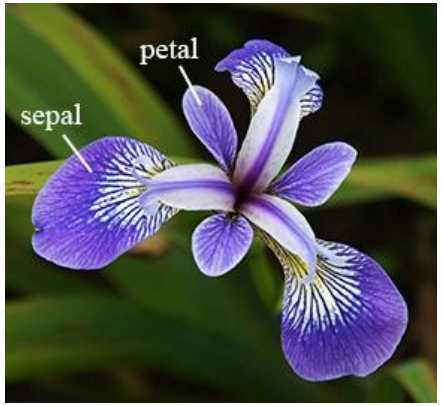
Grades	Encoded
A	4
B	3
C	2
D	1
Fail	0

`sklearn.preprocessing.` **`OneHotEncoder`**

One-Hot Encoding

Places
New York
Boston
Chicago
California
New Jersey

New York	Boston	Chicago	California	New Jersey
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1



Exercises

2.4 Perform various transformations, including standardization, normalization, and feature engineering.

We use the iris_cleaned.csv

a) Create a new feature petal_ratio (length divided by width) from the original features.

b) Standardize all numerical features including petal_ratio using z-score normalization. Verify the transformation by checking the mean and standard deviation.

c) Apply Min-Max scaling after standardization to all features. Verify the transformation by checking minimum and maximum values.

d) Compare the feature distributions: Original features, After standardization, After Min-Max scaling (applied after standardization)

What changes do you observe?

e) Does applying Min-Max scaling after standardization preserve zero mean? Why?

Recap

- Cleaning: Missing values, structural error, outliers, duplicates
- Aggregation
- Filtering
- Standardization/normalization
- Feature engineering

Questions ?